

Concepts

Positions

This guide explains how positions work in NautilusTrader, including their lifecycle, aggregation from order fills, profit and loss calculations, and the important concept of position snapshotting for netting OMS configurations.

Overview

A position represents an open exposure to a particular instrument in the market. Positions are fundamental to tracking trading performance and risk, as they aggregate all fills for a particular instrument and continuously calculate metrics like unrealized PnL, average entry price, and total exposure.

The system automatically creates positions when orders fill and tracks them from open to close. The platform supports both netting and hedging position management styles through its OMS (Order Management System) configuration.

Position lifecycle

Creation

The system opens a position on the first fill:

- **NETTING OMS:** Opens on first fill for an instrument (one position per instrument).
- **HEDGING OMS:** Opens on first fill for a new `position_id` (multiple positions per instrument).

A position tracks:

- Opening order and fill details.
- Entry side (`LONG` or `SHORT`).

- Initial quantity and average price.
- Timestamps for initialization and opening.



You can access positions through the Cache using `self.cache.position(position_id)` or `self.cache.positions(instrument_id=instrument_id)` from within your actors/strategies.

Updates

As additional fills occur, the position:

- Aggregates quantities from buy and sell fills.
- Recalculates average entry and exit prices.
- Updates peak quantity (maximum exposure reached).
- Tracks all associated order IDs and trade IDs.
- Accumulates commissions by currency.

Closure

A position closes when the net quantity becomes zero (`FLAT`). At closure:

- The closing order ID is recorded.
- Duration is calculated from open to close.
- Final realized PnL is computed.
- In `NETTING` OMS, when the position later reopens, the engine snapshots the closed state to preserve historical PnL (see [Position snapshotting](#)).

Order fill aggregation

Positions aggregate order fills to maintain an accurate view of market exposure. The aggregation process handles both sides of trading activity:

Buy fills

When a BUY order fills:

- Increases long exposure or reduces short exposure.
- Updates average entry price for opening trades.
- Updates average exit price for closing trades.
- Calculates realized PnL for any closed portion.

Sell fills

When a SELL order fills:

- Increases short exposure or reduces long exposure.
- Updates average entry price for opening trades.
- Updates average exit price for closing trades.
- Calculates realized PnL for any closed portion.

Net position calculation

The position maintains a `signed_qty` field representing the net exposure:

- Positive values indicate `LONG` positions.
- Negative values indicate `SHORT` positions.
- Zero indicates a `FLAT` (closed) position.

```
# Example: Position aggregation
# Initial BUY 100 units at $50
signed_qty = +100 # LONG position

# Subsequent SELL 150 units at $55
signed_qty = -50  # Now SHORT position

# Final BUY 50 units at $52
signed_qty = 0    # Position FLAT (closed)
```



Position adjustments

Position adjustments track quantity or PnL changes that occur outside of normal order fills, ensuring the position quantity accurately reflects the true net asset position. The system generates `PositionAdjusted` events for these scenarios.

Base currency commissions

When trading spot currency pairs (e.g., BTC/USDT) or FX spot, commissions paid in the base currency directly affect the net quantity received or delivered:

- **Opening fills:** Commission is deducted from the traded quantity. A buy of 1.0 BTC with 0.001 BTC commission results in a net long position of 0.999 BTC.
- **Closing fills:** Commission is applied to `signed_qty` because it affects actual inventory. Selling a 0.999 BTC LONG position with 0.000999 BTC commission leaves you SHORT 0.000999 BTC, not FLAT, because you gave up 0.999999 BTC total.
- **Flips:** Commission affects the final position size on both sides of the flip.

i Base currency commissions only apply to spot currency pairs and FX spot instruments where the commission currency matches `instrument.base_currency`. For other instruments, commissions are tracked separately and do not affect position quantity.

Funding payments

Funding adjustments track periodic payments for perpetual futures without affecting position quantity. These are logged with `quantity_change = None` and can include PnL impacts.

Adjustment tracking

All adjustments are preserved in the position event history:

- `position.adjustments` returns the list of all `PositionAdjusted` events.
- Each adjustment includes type (`COMMISSION` or `FUNDING`), quantity change, and timestamps.
- The adjustment history is cleared when positions close and reopen. When events are purged, commission adjustments tied to the removed fills are regenerated while non-commission adjustments (for example funding) are preserved.

OMS types and position management

NautilusTrader supports two primary OMS types that fundamentally affect how positions are tracked and managed. An `OmsType.UNSPECIFIED` option also exists, which defaults to the component's context. For full details, see the [Execution guide](#).

NETTING

In **NETTING** mode, all fills for an instrument are aggregated into a single position:

- One position per instrument ID.
- All fills contribute to the same position.
- Position flips from **LONG** to **SHORT** (or vice versa) as net quantity changes.
- Historical snapshots preserve closed position states.

HEDGING

In **HEDGING** mode, multiple positions can exist for the same instrument:

- Multiple simultaneous **LONG** and **SHORT** positions.
- Each position has a unique position ID.
- Positions are tracked independently.
- No automatic netting across positions.
- Closed positions remain in cache history but do not reopen; new fills create new positions.

 When using **HEDGING** mode, be aware of increased margin requirements as each position consumes margin independently. Some venues may not support true hedging mode and will net positions automatically.

Strategy vs venue OMS

The platform allows different OMS configurations for strategies and venues:

Strategy OMS	Venue OMS	Behavior
NETTING	NETTING	Single position per instrument at both strategy and venue.
HEDGING	HEDGING	Multiple positions supported at both levels.
NETTING	HEDGING	Venue tracks multiple, Nautilus maintains single position.
HEDGING	NETTING	Venue tracks single, Nautilus maintains virtual positions.



For most trading scenarios, keeping strategy and venue OMS types aligned simplifies position management. Override configurations are primarily useful for prop trading desks or when interfacing with legacy systems. See the [Live guide](#) for venue-specific OMS configuration.

Position snapshotting

Position snapshotting is an important feature for `NETTING` OMS configurations that preserves the state of closed positions for accurate PnL tracking and reporting.

Why snapshotting matters

In a `NETTING` system, when a position closes (becomes `FLAT`) and then reopens with a new trade, the position object is reset to track the new exposure. Without snapshotting, the historical realized PnL from the previous position cycle would be lost.

How it works

When a `NETTING` position closes and then receives a new fill for the same instrument, the execution engine snapshots the closed position state before resetting it, preserving:

- Final quantities and prices.
- Realized PnL.
- All fill events.
- Commission totals.

This snapshot is stored in the cache indexed by position ID. The position then resets for the new cycle while previous snapshots remain accessible. The Portfolio aggregates PnL across all snapshots for accurate totals.



This historical snapshot mechanism differs from optional position state snapshots (`snapshot_positions`), which periodically record open-position state for telemetry. See the [Live guide](#) for `snapshot_positions` and `snapshot_positions_interval_secs` settings.

Example scenario

```
# NETTING OMS Example
# Cycle 1: Open LONG position
```



```

BUY 100 units at $50 # Position opens
SELL 100 units at $55 # Position closes, PnL = $500
# Snapshot taken preserving $500 realized PnL

# Cycle 2: Open SHORT position
SELL 50 units at $54 # Position reopens (SHORT)
BUY 50 units at $52 # Position closes, PnL = $100
# Snapshot taken preserving $100 realized PnL

# Total realized PnL = $500 + $100 = $600 (from snapshots)

```

Without snapshotting, only the most recent cycle's PnL would be available, leading to incorrect reporting and analysis.

PnL calculations

NautilusTrader provides PnL calculations that account for instrument specifications and market conventions.

Realized PnL

Calculated when positions are partially or fully closed:

```

# For standard instruments
realized_pnl = (exit_price - entry_price) * closed_quantity * multiplier

# For inverse instruments (side-aware)
# LONG: realized_pnl = closed_quantity * multiplier * (1/entry_price - 1/exit_price)
# SHORT: realized_pnl = closed_quantity * multiplier * (1/exit_price - 1/entry_price)

```

The engine automatically applies the correct formula based on position side.

Unrealized PnL

Calculated using current market prices for open positions. The `price` parameter accepts any reference price (bid, ask, mid, last, or mark):

```

position.unrealized_pnl(last_price) # Using last traded price
position.unrealized_pnl(bid_price)  # Conservative for LONG positions
position.unrealized_pnl(ask_price)  # Conservative for SHORT positions

```

Returns `Money(0, settlement_currency)` for `FLAT` positions regardless of the price provided.

Total PnL

Combines realized and unrealized components:

```
total_pnl = position.total_pnl(current_price)
# Returns realized_pnl + unrealized_pnl
```



Currency considerations

- PnL is calculated in the instrument's settlement currency.
- For Forex, this is typically the quote currency.
- For inverse contracts, PnL may be in the base currency.
- Portfolio aggregates realized PnL per instrument in settlement currency.
- Multi-currency totals require conversion outside the Position class.

Commissions and costs

Positions track all trading costs:

- Commissions are accumulated by currency.
- Each fill's commission is added to the running total.
- Multiple commission currencies are supported.
- Realized PnL includes commissions only when denominated in the settlement currency.
- Other commissions are tracked separately and may require conversion.

```
commissions = position.commissions()
# Returns list[Money] with aggregated commission totals per currency

notional = position.notional_value(current_price)
# Returns Money in quote currency (standard) or base currency (inverse)
```



Limitations:

- Panics if inverse instrument has no `base_currency` set.
- Does not handle quanto contracts (returns quote currency instead of settlement currency).
- For quanto instruments, use `instrument.calculate_notional_value()` instead.

Position properties and state

Identifiers

- `id`: Unique position identifier.
- `instrument_id`: The traded instrument.
- `account_id`: Account where position is held.
- `trader_id`: The trader who owns the position.
- `strategy_id`: The strategy managing the position.
- `opening_order_id`: Client order ID that opened the position.
- `closing_order_id`: Client order ID that closed the position.

Position state

- `side`: Current position side (`LONG` , `SHORT` , or `FLAT`).
- `entry`: Direction of the currently open position (`Buy` for `LONG` , `sell` for `SHORT`). Updates when position flips direction.
- `quantity`: Current absolute position size.
- `signed_qty`: Signed position size (positive for `LONG` , negative for `SHORT`).
- `peak_qty`: Maximum quantity reached during position lifetime.
- `is_open`: Whether position is currently open.
- `is_closed`: Whether position is closed (`FLAT`).
- `is_long`: Whether position side is `LONG` .
- `is_short`: Whether position side is `SHORT` .

Pricing and valuation

- `avg_px_open`: Average entry price.
- `avg_px_close`: Average exit price when closing.
- `realized_pnl`: Realized profit/loss.
- `realized_return`: Realized return as decimal (e.g., 0.05 for 5%).

- `quote_currency` : Quote currency of the instrument.
- `base_currency` : Base currency if applicable.
- `settlement_currency` : Currency for PnL settlement.

Instrument specifications

- `multiplier` : Contract multiplier.
- `price_precision` : Decimal precision for prices.
- `size_precision` : Decimal precision for quantities.
- `is_inverse` : Whether instrument is inverse.

Timestamps

- `ts_init` : When position was initialized.
- `ts_opened` : When position was opened.
- `ts_last` : Last update timestamp.
- `ts_closed` : When position was closed.
- `duration_ns` : Duration from open to close in nanoseconds.

Associated data

- `symbol` : The instrument's ticker symbol.
- `venue` : The trading venue.
- `client_order_ids` : All client order IDs associated with position.
- `venue_order_ids` : All venue order IDs associated with position.
- `trade_ids` : All trade/fill IDs from venue.
- `events` : All order fill events applied to position.
- `event_count` : Total number of fill events applied.
- `last_event` : Most recent fill event.
- `last_trade_id` : Most recent trade ID.

 For complete type information and detailed property documentation, see the Position [API Reference](#).

Events and tracking

Positions maintain a complete history of events:

- All order fill events are stored chronologically.
- Associated client order IDs are tracked.
- Trade IDs from the venue are preserved.
- Event count indicates total fills applied.

This historical data enables:

- Detailed position analysis.
- Trade reconciliation.
- Performance attribution.
- Audit trails.

 Use `position.events` to access the full history of fills for reconciliation. The `position.trade_ids` property helps match against broker statements. See the [Execution guide](#) for reconciliation best practices.

Numerical precision

Position calculations use 64-bit floating-point (`f64`) arithmetic for PnL and average price computations. While fixed-point types (`Price` , `Quantity` , `Money`) preserve exact precision at configured decimal places, internal calculations convert to `f64` for performance and overflow safety.

Design rationale

The platform uses `f64` for position calculations to balance performance and accuracy:

- Floating-point operations are significantly faster than arbitrary-precision arithmetic.

- Raw integer multiplication can overflow even with 128-bit integers.
- Each calculation starts from precise fixed-point values, avoiding cumulative error.
- IEEE-754 double precision provides ~15 decimal digits of accuracy.

Validated precision characteristics

Testing confirms `f64` arithmetic maintains accuracy for typical trading scenarios:

- Standard amounts: No precision loss for amounts ≥ 0.01 in standard currencies.
- High-precision instruments: 9-decimal crypto prices preserved within $1e-6$ tolerance.
- Sequential fills: 100 fills show no drift (commission accuracy to $1e-10$).
- Extreme prices: Handles range from 0.00001 to 99,999.99999 without overflow.
- Round-trip trades: Opening and closing at same price produces exact PnL (commissions only).

For implementation details, see `test_position_pnl_precision_*` tests in `crates/model/src/position.rs`.

i For regulatory compliance or audit trails requiring exact decimal arithmetic, consider using `Decimal` types from external libraries. Very small amounts below `f64` epsilon ($\sim 1e-15$) may round to zero. This does not affect realistic trading scenarios with standard currency precisions (typically 2-9 decimals).

Integration with other components

Positions interact with several key components:

- **Portfolio**: Aggregates positions across instruments and strategies.
- **ExecutionEngine**: Creates and updates positions from fills.
- **Cache**: Stores position state and snapshots.
- **RiskEngine**: Monitors position limits and exposure.

i Positions are not created for spread instruments. While contingent orders can still trigger for spreads, they operate without position linkage. The engine handles spread instruments separately from regular positions.

Summary

Positions are central to tracking trading activity and performance. Understanding how positions aggregate fills, calculate PnL, and handle different OMS configurations matters when building trading strategies. Position snapshotting provides accurate historical tracking in `NETTING` mode, and the event history supports detailed analysis and reconciliation.

Related guides

- [Events](#) - How fills produce position events.
- [Orders](#) - Orders that create and modify positions.
- [Execution](#) - Fill handling that updates positions.
- [Portfolio](#) - Portfolio-level position aggregation.

[← Orders](#)[Previous Page](#)[Cache →](#)[Next Page](#)